

Inclusive design

Semantic HTML

Definizione: "semantico" si riferisce al **significato** dei tag. Un tag semantico dice al computer *che cosa* è quel pezzo di pagina, non solo come va disegnato.

Il problema di HTML4: si incapsulava tutto in `<div>` e `<span>`, che non significano niente (l'unica differenza è block vs inline). Il computer che elabora la pagina non sa cosa sia un div.

La soluzione di HTML5, i tag con significato:

Tag	Significato
<code><header></code> / <code><footer></code>	intestazione e piè di pagina
<code><nav></code>	navigazione
<code><main></code>	contenuto principale
<code><article></code>	un pezzo di contenuto autonomo
<code><aside></code>	barra laterale, contenuto complementare
<code><figure></code> / <code><figcaption></code>	grafici, diagrammi, immagini con didascalia

Esempio di pagina ben strutturata (dalla slide del corso):

```
<header>My Website</header>

<nav>

  <ol>

    <li><a href="/">Homepage</a></li>

    <li><a href="/me">About me</a></li>

  </ol>

</nav>

<main>

  <article>

    <h1>My favourite places</h1>

    <p>These are my favourite places to visit on holiday.</p>

  </article>

  <aside>

    <h2>Related blog posts</h2>

  </aside>

</main>

<footer>Thanks for visiting.</footer>
```

Perché conta: un utente vedente che vuole saltare il menu e arrivare al contenuto semplicemente scrolla. Uno screen reader come fa? Con i tag semantici **ottiene quel comportamento gratis**, perché stai dicendo al software come è fatta la pagina. Vale anche per l'HTML "vecchio": usare il type giusto negli input e mettere le label è già semantica.

Text

Le intestazioni. Con i CSS puoi far sembrare un `<h1>` minuscolo e un `<h6>` gigante. Quindi il tag **non va scelto in base all'aspetto**: scegli il tag per il significato semantico, poi lo stili come vuoi.

Le regole:

- **un solo `<h1>` per pagina**, ed è il titolo della pagina, la cosa che ti dice di cosa parla tutto il resto;
- sotto, **più `<h2>`** per gli argomenti principali;
- dentro ciascuno, `<h3>`, `<h4>` ecc. per le sottosezioni;
- **gerarchia senza salti**: mai passare da h2 a h4.

Perché i salti sono un problema (esempio concreto): se navighi con lo screen reader per titoli e senti "intestazione livello 4" subito dopo un livello 2, pensi di **esserti perso l'h3** e torni indietro a cercarlo. Ma non c'è mai stato. È molto confusionario.

Caso tipico: in una pagina articolo, l'h1 è il nome dell'articolo; sidebar e footer, se hanno un titolo, sono anche loro h2; le sottosezioni scendono a h3/h4.

Il layout del testo:

- **contrasto forte** (vedi lezione Colour);
- **testo grande**;
- usare i tag `<p>` e non i `<br>` per separare i paragrafi;
- curare **interlinea e scelta del font**.

Sui font: i **serif** (quelli con le grazie, le piccole decorazioni terminali) tradizionalmente sono considerati più leggibili per la maggior parte delle persone, ma ci sono evidenze che per condizioni come l'autismo siano **più difficili** da leggere. Quindi i **sans-serif** sono un'ottima scelta per il corpo del testo, eventualmente tenendo i serif per i titoli.

L'esempio visivo della slide: a sinistra un unico paragrafo enorme senza interlinea; a destra lo stesso testo spezzato in due paragrafi con interlinea ampia. Nessuno discute su quale sia più leggibile. E qui sta il messaggio del modulo: *non stai aiutando solo una condizione specifica, stai rendendo il sito migliore per tutti.*

Links

La regola fondamentale — usare il tag giusto per lo scopo giusto:

Tag	Quando usarlo	Quando NON usarlo
<code><a></code>	per portare l'utente a un'altra pagina	con URL finti e handler onClick

Tag	Quando usarlo	Quando NON usarlo
<code><button></code>	per eseguire un'azione (inviare un form, cambiare qualcosa nella pagina)	come semplice link a un'altra pagina

L'anti-pattern da evitare (letteralmente dalla slide):

```
<a href="#" onClick="javascript:doSomething();">Click me</a>
```

Qui hai preso un link, gli hai messo un URL finto (#) e lo hai dirottato con un evento click per fargli fare tutt'altro. Va usato un `<button>`. Per lo stile: con i CSS puoi far sembrare un link un pulsante e viceversa, quindi **prima la semantica, poi l'estetica**.

Indicare i link con più del colore. Tradizionalmente i link sono blu e **sottolineati**: la sottolineatura è il secondo indicatore, quello che funziona anche per i daltonici. Stesso discorso per i **link visitati**: lo screen reader annuncia "visited link", quindi se anche visivamente li distingui dai non visitati dall'utente ipovedente la stessa informazione che ottiene chi usa uno screen reader.

Testo descrittivo, non "clicca qui". Lo screen reader legge una sezione alla volta e annuncia "link" seguito dal testo del link. Se hai 10 link che dicono tutti "clicca qui", l'utente sente "link clicca qui, link clicca qui, link clicca qui" e non ha idea di dove portino. Stesso identico problema per la **SEO**: anche i motori di ricerca non capiscono di cosa parla il link.

Colour

Il contrasto. Se metti testo nero su bianco è facilissimo da leggere; il problema nasce quando metti testo sopra **immagini**, dove il contrasto cambia da zona a zona.

Definizione di rapporto di contrasto: rapporto numerico tra la luminanza del testo e quella dello sfondo. Le soglie WCAG sono:

AA AAA

Testo normale **4.5:1 7:1**

Testo grande **3:1 4.5:1**

Il testo grande può permettersi meno contrasto perché è già più facile da leggere.

Come si misura: con un contrast checker, per esempio quello di WebAIM (webaim.org/resources/contrastchecker/), dove inserisci i due valori esadecimali. Esistono anche plugin del browser che te lo calcolano direttamente sulla pagina, senza dover estrarre i codici colore a mano.

Esempio del corso: blu scuro su bianco → ottimo contrasto, va bene ovunque. Rosso medio su bianco → **non** passa le linee guida per il testo normale, passa a stento per il testo grande, e non raggiunge comunque il AAA.

Il colore non deve mai essere l'unico indicatore. Non tutti vedono i colori.

L'esempio del messaggio di errore (il più chiaro del corso). Versione sbagliata: il messaggio di errore è scritto solo in rosso. Per un daltonico rosso/verde quel testo non comunica "errore". Versione corretta: si aggiunge un **triangolo di attenzione** con il relativo **alt text**, così l'utente riceve l'informazione da **quattro canali** — il colore, l'icona, il testo descrittivo dell'icona e il testo del messaggio. Se uno di questi canali è compromesso, restano gli altri.

Chris aggiunge un argomento culturale: non è scontato che il rosso significhi errore, è una convenzione che varia da cultura a cultura. Dare più contesto semantico possibile rende tutto più chiaro.

Images

Definizione di attributo `alt`: descrizione testuale alternativa dell'immagine, che gli screen reader leggono al posto dell'immagine stessa.

Immagine informativa — l'alt descrive il contenuto:

```

```

Immagine decorativa — l'alt va lasciato **vuoto**, ma deve esserci:

```

```

Perché vuoto e non assente? Questa è la distinzione più importante della lezione. Se **ometti** l'attributo, lo screen reader annuncia *"immagine non etichettata"* e l'utente si chiede cosa si stia perdendo. Se metti `alt=""`, lo screen reader sa che è decorativa e la **ignora silenziosamente**.

Esempio del corso: dei pulsanti con una piccola icona di busta accanto al testo "Iscriviti alla nostra newsletter" e un'icona di persona accanto a "Richiedi un richiamo". Le icone non aggiungono niente rispetto al testo del link, quindi descriverle sarebbe solo ripetizione: alt vuoto.

Testo dentro le immagini: da evitare. Motivi: gli screen reader non possono leggerlo, e gli strumenti di accessibilità che modificano lo stile del testo (aumentare il contrasto, ingrandire il font) non possono agire su pixel. Se proprio devi, il testo nell'immagine deve rispettare gli standard di contrasto (almeno 3:1, e di più se è testo piccolo o se punti al AAA).

Audio & Video

In produzione:

- **evitare immagini lampeggianti.** Causano problemi reali a chi ha **epilessia**, danno fastidio a chi ha autismo, e in generale infastidiscono tutti;
- **registrare le audio description mentre giri il video**, perché rifarle dopo è molto difficile;
- se devi mettere del testo nel video, che sia **grande e ad alto contrasto**.

Nel player:

- usare i **tag nativi** `<video>` e `<audio>` invece di player proprietari o vecchi player Flash. Motivo: uno strumento di accessibilità può ispezionare l'HTML, capire cosa sta succedendo e dare all'utente il controllo; con un player proprietario non ci arriva;
- **niente autoplay.** È una linea guida di accessibilità, ma anche tutti gli altri ti ringrazieranno;
- se implementi **controlli personalizzati**, testali con la navigazione da tastiera e con uno screen reader.

Definizioni da tenere distinte:

- **Trascrizione:** la versione scritta e curata di tutto il contenuto parlato, separata dal video.
- **Captions (didascalie):** i sottotitoli sincronizzati che appaiono *dentro* il video, e che descrivono sia ciò che viene detto sia ciò che accade.
- **Subtitles (sottotitoli):** attenzione, nella slide Chris precisa che tecnicamente si riferiscono alla **traduzione in lingua straniera** e non includono le descrizioni dei suoni.

Perché la trascrizione conviene a tutti: anche i normoudenti spesso preferiscono leggere — sei sui mezzi pubblici, hai dimenticato le cuffie, o semplicemente ti piace leggere mentre guardi. Chris cita i corsi **edX**, che mettono il video da un lato e la trascrizione dall'altro. In più (dalla slide) **la trascrizione rende il contenuto ricercabile**.

Come ottenerle concretamente: servizi a pagamento come **Rev** o freelance su **Fiverr**; caricare il video su **YouTube**, che genera le didascalie automaticamente; oppure farle a mano con software tipo **Subs Factory**. Dipende dal budget.

Forms

Le label. Ogni `<input>` deve essere abbinato a una `<label>`, collegata tramite l'attributo `for` che punta all'id dell'input:

```
<label for="userId">User ID</label>
```

```
<input type="text" id="userId" name="userId" />
```

Così, quando lo screen reader arriva al campo, legge l'etichetta e l'utente sa cosa scrivere.

I placeholder non sostituiscono le label.

Definizione di placeholder: testo grigio che appare dentro il campo vuoto e sparisce quando inizi a scrivere.

Il placeholder deve essere un **suggerimento aggiuntivo**, non la stessa informazione della label.

Esempio: per un campo data di nascita, la label dice "Data di nascita", e il placeholder — se lo usi — dice GG/MM/AAAA, cioè il **formato richiesto**. Mettere "Data di nascita" nel placeholder è duplicazione inutile.

Chris aggiunge un'osservazione dalla sua **tesi di psicologia**, dove aveva studiato proprio l'effetto dei placeholder: confondono spesso le persone, perché sembrano un valore già inserito o un autocompletamento. Gli utenti a volte **saltavano del tutto il campo** pensando fosse già compilato. La sua conclusione è che spesso conviene **non usarli affatto**.

Non disabilitare i pulsanti.

Definizione di tab index: l'ordine in cui gli elementi interattivi (link, input, bottoni) ricevono il focus quando premi Tab.

Se metti l'attributo `disabled` su un pulsante, quel pulsante **esce dal tab index** e di fatto **scompare** per chi naviga da tastiera o con screen reader: l'utente tabula per tutto il form, arriva alla fine e non trova il bottone di invio. Si confonde e non capisce dove sia.

L'approccio corretto: lascia il pulsante attivo, permetti all'utente di premerlo, e **poi digli cosa c'è che non va**.

La gestione degli errori è la parte più difficile. Le regole:

- portare l'utente **alla fonte dell'errore**;
- rendere chiarissimo **quali campi specifici** sono sbagliati. L'anti-esempio è il classico "Compila tutti i campi obbligatori": alcuni sono opzionali, altri no, e non sai quale hai sbagliato;
- associare il messaggio d'errore al campo con **aria-describedby**.

Esempio del corso: un campo email dove l'indirizzo inserito risulta già registrato. La `<label>` dice cos'è il campo ("Indirizzo email"); l'`aria-describedby` punta al messaggio che dice "questo indirizzo è già registrato, effettua il login o usane un altro".

ARIA

Definizione: **WAI-ARIA** sta per *Web Accessibility Initiative – Accessible Rich Internet Applications*. È un insieme di attributi creato dal W3C e incorporato in HTML, nato quando abbiamo iniziato a costruire nel browser applicazioni complesse in stile desktop, per le quali non esisteva un modo di trasmettere il significato semantico alle tecnologie assistive.

La regola d'oro: ARIA **non sostituisce** l'HTML semantico. L'HTML semantico è sempre la prima scelta; ARIA si usa quando non riesci a esprimere il significato con l'HTML puro.

L'attributo `role`. Serve a dire alla tecnologia assistiva *che tipo di elemento* è, quando stai simulando un elemento nativo con altri tag.

- `role="navigation"` se stai costruendo un menu custom senza `<nav>`;
- `role="checkbox"` se hai stilizzato una checkbox con una serie di `<span>` annidati (cosa comune con i form "fancy"), così l'assistiva capisce che quello è una checkbox anche se non è un `<input type="checkbox">`;
- `role="text"`: caso controverso, perché **non fa parte delle specifiche ufficiali**, ma molti browser lo implementano. Serve quando i tag annidati dentro un elemento fanno sì che lo screen reader spezzetti la frase in modo innaturale: mettendolo sull'elemento genitore, il più delle volte la legge di seguito. Chris ammette che molti consulenti preferiscono attenersi allo standard, lui è più pragmatico e privilegia l'esperienza reale di chi usa lo screen reader.

Evitare la ridondanza: se usi `<nav>` **non** devi aggiungere `role="navigation"`, è già implicito. `role` serve solo quando stai rimpiazzando un elemento con tag diversi.

L'esempio W3C delle tab. Le schede sono costruite con `<div>` perché non esiste un tag `<tab>` in HTML: al contenitore si dà `role="tablist"`, ai pulsanti `role="tab"`, si usa `aria-selected` per indicare quale è attiva e quale pannello controlla.

Il `tabindex` spiegato:

Valore	Effetto
numero positivo (es. 5)	forza l'elemento a una posizione specifica nell'ordine di tabulazione
0	inserisce l'elemento nel tab index seguendo il normale flusso della pagina (è quello che vuoi quasi sempre)

	Valore	Effetto
-1		l'elemento non è raggiungibile con Tab, ma puoi dargli il focus manualmente via JavaScript

Gli attributi **aria**-:

- **aria-label** — aggiunge un testo a un elemento che non ne ha. Esempio classico: l'**icona hamburger** del menu. È ovvia per chi vede, non per chi no. `aria-label="Menu"`.
- **aria-hidden** — rimuove dall'albero di accessibilità tutto ciò che è puramente presentazionale. **Esempio del corso:** un articolo composto da un'immagine e un titolo, dove l'immagine dice esattamente la stessa cosa del titolo. Chi usa lo screen reader non guadagna niente a sapere che quell'immagine esiste, quindi la si nasconde. (Variante: se l'immagine è avvolta in un link, si può nascondere l'intero link, visto che l'informazione è già nel titolo sotto.)
- **aria-checked** — indica lo stato di una checkbox simulata con uno `<span>`.
- **aria-disabled** — la soluzione al problema del pulsante disabilitato visto nella lezione Forms: stili il pulsante con i CSS perché **sembri** disabilitato (grigio, cursore con la X all'hover), ma **non** usi l'attributo `disabled`, così resta nel tab index; con `aria-disabled` comunichi allo screen reader la stessa informazione visiva che riceve l'utente vedente.
- **aria-labelledby** e **aria-describedby** — collegano un elemento ad altri elementi della pagina tramite il loro `id`. La differenza pratica: **labelledby viene letto subito, all'inizio**, perché spiega *cos'è* il campo; **describedby viene letto dopo, alla fine**, perché è informazione supplementare. Esempio: uno `<span>` con `role="checkbox"` per l'iscrizione alla newsletter, `aria-labelledby` che punta al titolo "Iscriviti alla newsletter" e `aria-describedby` che punta al testo piccolo sotto con i dettagli.

Le live region.

Definizione: una zona della pagina marcata con `aria-live`, che dice allo screen reader di annunciare i cambiamenti che avvengono lì dentro dopo il caricamento.

Serve perché nelle applicazioni moderne i contenuti si aggiornano dinamicamente: se sei in fondo alla pagina e qualcosa cambia in cima, senza `aria-live` non lo sapresti mai.

Due modalità:

- **polite** — lo screen reader annuncia il cambiamento **quando ha finito** quello che sta leggendo;
- **assertive** — lo annuncia **immediatamente**, interrompendo.

Per errori e avvisi esiste una scorciatoia: `role="alert"`, che fa leggere subito il messaggio.